MODULE 9

AIOps: CI/CD & Release Management

Duration	Format	Audience	Day / Position
45 min	**Lecture + Demo**	**Mixed Engineers**	**Day 2 / First**

LEARNING OBJECTIVES

By the end of this module, participants will be able to:

1. Integrate Claude Code into GitHub Actions CI/CD pipelines using headless mode (claude -p) for automated code review and testing
2. Build automated lint/test pipelines with AI-powered failure explanation and suggested fixes
3. Generate professional release notes, changelogs, and PR quality checks using Claude Code automation
4. Configure Claude Code hooks (pre-commit and post-push) for automated quality enforcement
5. Design a complete CI workflow for the TI sensor toolkit project built in Lab 1
6. Apply AIOps principles to reduce manual DevOps toil and accelerate release cycles

SESSION TIMING BREAKDOWN

0:00 – 0:10	**Section 1: Claude Code in GitHub Actions CI/CD**
0:10 – 0:20	**Section 2: AI-Powered Pipeline Failure Analysis**
0:20 – 0:28	**Section 3: Automated Release Notes & PR Quality**
0:28 – 0:37	**Section 4: Hooks — Pre-Commit & Post-Push Automation**
0:37 – 0:43	**Section 5: Practical — CI Workflow for Sensor Toolkit**
0:43 – 0:45	**Wrap-Up: Quick Reference & Transition to Module 10**

0:00 – 0:10	SECTION 1: Claude Code in GitHub Actions CI/CD

1. Claude Code in GitHub Actions CI/CD

1.1 Welcome to Day 2

Welcome back! Yesterday you built a Python project from scratch, learned to refactor and debug, and mastered AI-driven testing. Today we connect Claude Code to real-world DevOps systems. Module 9 is about making Claude Code an integral part of your CI/CD pipeline — not just a developer tool, but a team-wide automation layer.

1.2 Why Claude Code in CI/CD?

Without Claude in CI	With Claude in CI
Developer pushes code, CI runs lint + tests	Same, plus: Claude reviews the diff for bugs, security, style violations
Build fails: developer reads log, debugs manually	Build fails: Claude explains the failure and suggests a fix in the PR comment
Release: developer writes changelog manually	Release: Claude generates release notes from git log automatically
PR review: wait for human reviewer (hours/days)	PR review: Claude provides instant first-pass review, human reviewer focuses on architecture

1.3 GitHub Actions Integration

Claude Code runs in CI/CD using headless mode (claude -p) with JSON output. Here's the fundamental pattern:

```
# .github/workflows/claude-review.yml
name: AI Code Review
on: [pull_request]

jobs:
  ai-review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0  # Full history for diff

      - name: Install Claude Code
        run: npm install -g @anthropic-ai/claude-code

      - name: Run AI Code Review
        env:
          ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
        run: |
          git diff origin/main...HEAD | claude -p \
            "Review this PR diff for bugs, security issues, \
             and style violations. Rate each: critical/major/minor." \
            --output-format json \
```

```
            --allowedTools "Read,Grep" > review.json

      - name: Post Review Comment
        uses: actions/github-script@v7
        with:
          script: |
            const review = require('./review.json');
            await github.rest.issues.createComment({
              ...context.repo,
              issue_number: context.issue.number,
              body: `## AI Review\n${review.result}`
            });
```

☆ **KEY POINT:** The ANTHROPIC_API_KEY must be stored as a GitHub secret. Never hardcode API keys in workflow files. In a TI environment, work with your DevOps team to add the secret at the organization or repository level.

0:10 – 0:20	SECTION 2: AI-Powered Pipeline Failure Analysis

2. AI-Powered Pipeline Failure Analysis

2.1 The Failure Explainer Pattern

When a CI pipeline fails, developers waste 10–30 minutes reading logs, identifying the root cause, and figuring out a fix. Claude Code can analyze failure logs and explain the problem in seconds:

```
# Add to your CI workflow after a failed step:
- name: Analyze Test Failure
  if: failure()
  env:
    ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
  run: |
    # Capture the test output from the failed step
    pytest tests/ -v 2>&1 | tail -100 > test-output.txt

    # Ask Claude to explain and suggest a fix
    cat test-output.txt | claude -p \
      "This CI test run failed. Explain:
       1. Which test(s) failed and why
       2. Root cause analysis
       3. Suggested fix (with code if possible)
       4. Is this a flaky test or a real bug?" \
      --output-format json > failure-analysis.json
```

2.2 Lint Failure Auto-Fix

For lint and formatting failures, Claude can not only explain the issue but propose an auto-fix PR:

```
# Auto-fix lint failures in CI:
- name: Auto-Fix Lint Issues
  if: failure()
  run: |
    # Get the lint output
    ruff check src/ --output-format json > lint-errors.json

    # Ask Claude to fix them
    claude -p "Fix all ruff violations listed in \
      @lint-errors.json. Apply minimal changes." \
      --allowedTools "Read,Write"

    # Create a fix branch and PR
    git checkout -b fix/auto-lint-$(date +%s)
    git add -A && git commit -m "fix: auto-fix lint violations"
    git push origin HEAD
    gh pr create --title "Auto-fix: lint violations" \
      --body "Automated lint fixes by Claude Code"
```

Track A (IT Engineers)	Track B (Process Engineers)

Add failure analysis to existing CI/CD pipelines. Integrate with Slack/Teams for alerts.	Set up a simple GitHub Actions workflow that runs pytest on push and explains any failures.
Build auto-fix pipelines for lint + test failures with automatic PR creation.	Use Claude's failure explanation to understand errors faster. Focus on reading the analysis, not building the pipeline.
Add deployment validation: Claude reviews staging logs after deploy.	Add data validation step: Claude checks CSV files on push to data/ directory.

⚠ **WARNING:** Auto-fix pipelines should NEVER commit directly to main. Always create a separate branch and PR that requires human review. The goal is to speed up the fix process, not bypass code review.

0:20 – 0:28	SECTION 3: Automated Release Notes & PR Quality

3. Automated Release Notes & PR Quality Checks

3.1 AI-Generated Release Notes

Release notes are tedious to write manually but essential for team communication. Claude Code can generate them from git history:

```
# Generate release notes from git log:
git log --oneline v1.0.0..HEAD | claude -p \
  "Generate professional release notes from these commits.
   Group into: Features, Bug Fixes, Breaking Changes,
   Internal. Use Markdown format with bullet points.
   Start with a one-line summary of this release." \
  > RELEASE_NOTES.md

# Automate in GitHub Actions on tag push:
name: Release Notes
on:
  push:
    tags: ['v*']
jobs:
  release:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }
      - run: npm install -g @anthropic-ai/claude-code
      - run: |
          PREV_TAG=$(git describe --tags --abbrev=0 HEAD^)
          git log --oneline $PREV_TAG..HEAD | claude -p \
            "Generate release notes" > notes.md
      - uses: softprops/action-gh-release@v2
        with:
          body_path: notes.md
```

3.2 PR Quality Checks

```
# PR quality checklist enforced by Claude:
- name: PR Quality Check
  run: |
    PR_BODY=$(gh pr view ${{ github.event.number }} --json body -q .body)
    DIFF=$(git diff origin/main...HEAD)

    echo "$DIFF" | claude -p \
      "Check this PR against quality criteria:
       1. Does the diff match the PR description?
       2. Are there new functions without tests?
       3. Are there hardcoded values that should be config?
       4. Is the diff size reasonable (<500 lines)?
       Output: PASS or FAIL with reasons." \
      --output-format json > quality.json

    # Fail the check if quality issues found
```

```
    if jq -e '.result | test("FAIL")' quality.json; then
      echo '::error::PR quality check failed'
      exit 1
    fi
```

☆ **KEY POINT:** AI-generated release notes save 30–60 minutes per release cycle. PR quality checks catch common issues (missing tests, large diffs, hardcoded values) before human reviewers see the PR, letting reviewers focus on architecture and logic.

0:28 – 0:37	SECTION 4: Hooks — Pre-Commit & Post-Push Automation

4. Hooks: Pre-Commit & Post-Push Automation

4.1 Claude Code Hooks

Claude Code supports hooks that run automatically before or after specific events. Hooks are configured in .claude/settings.json and execute shell commands at trigger points.

Hook Event	When It Runs	Use Case
PreToolUse	Before Claude runs a tool (Read, Write, Bash)	Validate file access, block writes to protected dirs
PostToolUse	After a tool completes	Log all changes, validate output format
Notification	When Claude sends a notification	Forward to Slack, update dashboard
Stop	When Claude finishes a task	Run linter, commit changes, send report

```
// .claude/settings.json - Hook configuration
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write",
        "hooks": [
          {
            "type": "command",
            "command": "ruff check $CLAUDE_FILE_PATH 2>/dev/null || true"
          }
        ]
      }
    ],
    "Stop": [
      {
        "matcher": "",
        "hooks": [
          {
            "type": "command",
            "command": "pytest tests/ -q --tb=no 2>/dev/null || true"
          }
        ]
      }
    ]
  }
}
```

4.2 Pre-Commit Hooks with Claude Code

Integrate Claude Code into your existing pre-commit workflow to catch issues before they reach CI:

```
#!/bin/bash
# .git/hooks/pre-commit (or via pre-commit framework)
```

```
# Get staged files
STAGED=$(git diff --cached --name-only --diff-filter=ACM | grep '\.py$')

if [ -n "$STAGED" ]; then
  # Quick lint check with Claude
  git diff --cached | claude -p \
    "Check this diff for: syntax errors, missing imports,
     obvious bugs, and style violations. Output: PASS or
     FAIL with line numbers." \
    --allowedTools "Read,Grep" \
    --output-format json > /tmp/precommit-check.json

  if echo $(cat /tmp/precommit-check.json) | jq -e '.result | test("FAIL")'; then
    echo "Pre-commit: Claude found issues. Review above."
    jq -r '.result' /tmp/precommit-check.json
    exit 1  # Block the commit
  fi
fi
```

⚠ **WARNING:** Pre-commit hooks with Claude add ~5-10 seconds per commit (API latency). For teams that commit frequently, consider running Claude only on files over a certain size or only before push, not every commit. Use timeout 15 claude -p to prevent blocking.

0:37 – 0:43	SECTION 5: Practical — CI Workflow for Sensor Toolkit

5. Practical: CI Workflow for Sensor Toolkit

Let's build a complete GitHub Actions workflow for the sensor toolkit project from Lab 1. This ties together everything from Module 9:

```
# .github/workflows/sensor-toolkit-ci.yml
name: Sensor Toolkit CI
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  quality:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.10' }
      - run: pip install -r requirements-dev.txt

      # Step 1: Lint
      - name: Ruff lint
        run: ruff check src/ tests/

      # Step 2: Format check
      - name: Black format check
        run: black --check src/ tests/

      # Step 3: Tests with coverage
      - name: Pytest with coverage
        run: pytest tests/ -v --cov=src/sensor_toolkit
                --cov-report=xml --cov-fail-under=70

  ai-review:
    if: github.event_name == 'pull_request'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }
      - run: npm install -g @anthropic-ai/claude-code
      - name: AI Code Review
        env:
          ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
        run: |
          git diff origin/main...HEAD | claude -p \
            "Review for bugs, security, and TI coding standards." \
            --output-format json --allowedTools "Read,Grep" \
            > review.json
          echo "## AI Review" > comment.md
          jq -r '.result' review.json >> comment.md
```

✅ COMPLETE CI PIPELINE CHECKLIST

- Lint: ruff check catches style violations
- Format: black --check enforces consistent formatting
- Tests: pytest with coverage threshold (70%+ minimum)
- AI Review: Claude reviews PR diffs for bugs and security (on PRs only)
- Failure Analysis: Claude explains test failures (on failure)
- Release Notes: Claude generates from git log (on tag push)

0:43 – 0:45	WRAP-UP: Quick Reference & Transition to Module 10

6. Quick Reference & Transition

MODULE 9 RECAP

✅ GitHub Actions + Claude Code: install via npm, use claude -p with ANTHROPIC_API_KEY secret, post results as PR comments.

✅ Failure analysis: pipe test/build output to Claude, get root cause + suggested fix. Add as if: failure() step.

✅ Release notes: pipe git log to Claude, generate grouped Markdown. Automate on tag push.

✅ PR quality checks: Claude validates diff vs. description, missing tests, hardcoded values, diff size.

✅ Hooks: .claude/settings.json configures PostToolUse (auto-lint) and Stop (auto-test) hooks.

✅ Pre-commit: git diff --cached | claude -p for quick issue detection before commit.

QUICK REFERENCE CARD

Automation	Implementation	When
AI code review	gh pr diff | claude -p "Review for bugs" --output-format json	On every PR
Failure explainer	pytest 2>&1 | claude -p "Explain this failure"	On test failure
Auto-fix lint	ruff output | claude -p "Fix violations" --allowedTools Write	On lint failure
Release notes	git log v1..HEAD | claude -p "Generate release notes"	On tag push
PR quality check	git diff | claude -p "Check against criteria"	On every PR
Pre-commit hook	git diff --cached | claude -p "Check for issues"	Before each commit
PostToolUse hook	.claude/settings.json: ruff check on Write	After every file write
Stop hook	.claude/settings.json: pytest on task completion	After Claude finishes

TRANSITION TO MODULE 10

You now know how to put Claude Code to work in your CI/CD pipeline. In Module 10 (Agents as Your Team: Foundations), you'll learn about Claude Code subagents — specialized AI agents that play different roles (planner, critic, executor) and can be configured via YAML files. This is where Claude Code goes from being a tool to being a team member.